End-Semester Examination Study Guide: SQLite3 Database Operations, NumPy, and Pandas (100 Marks)

Part 1: Theoretical Foundations & Architecture

1. **SQLite3 Relational Database Engine**:
   - **Lightweight & Embedded**: SQLite3 is a lightweight, file-based relational database management system that requires no separate server process, server setup, or user authentication. The entire database resides in a single file on disk (e.g., `college.db`).
   - **Python Standard Module**: Python includes the `sqlite3` module built into its standard library, requiring no external installation.
   - **The 5-Step Execution Workflow**:
     1. `import sqlite3` & `conn = sqlite3.connect('filename.db')` — Establishes a database connection handle (and creates the database file if it does not exist).
     2. `cursor = conn.cursor()` — Creates a cursor object to execute SQL commands and traverse query result sets.
     3. `cursor.execute(sql_query, params)` — Executes DDL/DML statements (`CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`).
     4. `conn.commit()` — Saves and commits all data modifications permanently to the database file.
     5. `conn.close()` — Safely closes the database handle to prevent file locking.
   - **Parameterized Queries (`?` Placeholders)**: Values must always be passed into SQL execution methods using `?` placeholders paired with a tuple of arguments (e.g., `cursor.execute("SELECT * FROM students WHERE branch = ?", (branch,))`). Never concatenate strings or use f-strings to format SQL queries, as doing so exposes the database to critical **SQL Injection** vulnerabilities.
   - **Fetching Methods**:
     - `cursor.fetchone()`: Retrieves the next single row tuple from a result set, or `None` if no rows remain.
     - `cursor.fetchmany(n)`: Retrieves the next (n) rows as a list of tuples.
     - `cursor.fetchall()`: Retrieves all remaining matching rows as a list of tuples.

2. **NumPy (Numerical Python)**:
   - **Vectorized Array Computation**: NumPy provides the core (N)-dimensional array object (`ndarray`) for fast numerical computation. Unlike standard Python lists that require

explicit `for` loops for element-wise calculations, NumPy arrays perform **vectorized operations** compiled directly in low-level memory.

3. **Pandas (Data Analysis Library)**:
    - **Structured Data Containers**: Built on top of NumPy, Pandas provides two primary data structures: **Series** (1-dimensional labeled array) and **DataFrame** (2-dimensional tabular structure with labeled rows and columns).
    - **Tabular Operations**: Supports data ingestion (CSVs, Excel files, and SQL database tables), indexing, conditional filtering, missing value imputation, and group aggregation.

Part 2: 100-Mark End-Semester Exam Suite (5 × 20M)

Question 1 (20 Marks): SQLite3 Relational Operations & Parameterized Query Safety

(a) [5 Marks] Theoretical Concepts

1. List the 5 fundamental steps of the Python `sqlite3` execution workflow.
2. Explain the purpose of `?` placeholders in `cursor.execute()` and why f-string SQL query construction must be avoided.

(b) [5 Marks] Result Set Retrieval Methods

Differentiate between `cursor.fetchone()`, `cursor.fetchmany(n)`, and `cursor.fetchall()`.

© [5 Marks] Simple Baseline Solution: SQLite Table Creation & Insertion

```
import sqlite3

# 1. Connect and create database handle
conn = sqlite3.connect("academic.db")
cursor = conn.cursor()

# 2. Create Table
cursor.execute('''
CREATE TABLE IF NOT EXISTS students (
    roll_no INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    marks REAL
)
```

```
''')
conn.commit()

# 3. Single & Batch Insertion
cursor.execute("INSERT INTO students VALUES (?, ?, ?)", (101, "Alpha", 88.5))

batch_students = [
    (102, "Beta", 92.0),
    (103, "Gamma", 75.5)
]
cursor.executemany("INSERT INTO students VALUES (?, ?, ?)", batch_students)
conn.commit()

# 4. Query Data
cursor.execute("SELECT * FROM students WHERE marks >= ?", (80.0,))
records = cursor.fetchall()
for row in records:
    print("Student Record:", row)

# 5. Close Connection
conn.close()
```

(d) [5 Marks] Complex Improvement: Modular OOP SQLite Manager with Resource Cleanup & Exception Handling

This improved solution uses Object-Oriented Programming, custom exceptions, parameterized query safety, and a `try/except/finally` structure to ensure connection handles close safely.

```
import sqlite3

# --- Custom Database Exception ---
class DatabaseOperationError(Exception):
    """Custom exception raised when a database DDL/DML operation fails."""
    def __init__(self, message, original_error=None):
        self.message = message
        self.original_error = original_error
        super().__init__(f"Database Failure: {message} | Details: {original_error}")

# --- Modular Database Engine Class ---
class StudentDatabaseManager:
    def __init__(self, db_filepath="academic_system.db"):
        self.db_filepath = db_filepath

    def get_connection(self):
        """Establishes and returns a database connection instance."""
```

```
        return sqlite3.connect(self.db_filepath)

    def initialize_schema(self):
        """Creates table schema inside a safe connection context."""
        conn = None
        try:
            conn = self.get_connection()
            cursor = conn.cursor()
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS student_records (
                    student_id INTEGER PRIMARY KEY,
                    student_name TEXT NOT NULL,
                    department TEXT NOT NULL,
                    cgpa REAL CHECK(cgpa >= 0.0 AND cgpa <= 10.0)
                )
            ''')
            conn.commit()
            print("[Database] Schema initialized successfully.")
        except sqlite3.Error as err:
            raise DatabaseOperationError("Failed to initialize database schema.", err
        finally:
            if conn:
                conn.close()  # Guarantees resource cleanup

    def add_student(self, student_id, name, department, cgpa):
        """Safely inserts a single student record using parameterized queries."""
        conn = None
        try:
            # Input Type Validation
            if not isinstance(student_id, int) or student_id <= 0:
                raise ValueError("Student ID must be a positive integer.")
            if not isinstance(cgpa, (int, float)) or not (0.0 <= cgpa <= 10.0):
                raise ValueError("CGPA must be a float between 0.0 and 10.0.")

            conn = self.get_connection()
            cursor = conn.cursor()

            # Parameterized Query
            cursor.execute(
                "INSERT INTO student_records (student_id, student_name, department, c
                (student_id, name.strip(), department.strip(), float(cgpa))
            )
            conn.commit()
            print(f"[Database] Student '{name}' inserted successfully.")

        except sqlite3.IntegrityError as err:
```

```
                raise DatabaseOperationError(f"Duplicate entry for Student ID {student_id
            except sqlite3.Error as err:
                raise DatabaseOperationError("Insertion failed.", err)
            finally:
                if conn:
                    conn.close()

        def fetch_students_by_dept(self, dept_name):
            """Retrieves matching records using parameterized SELECT queries."""
            conn = None
            try:
                conn = self.get_connection()
                cursor = conn.cursor()
                cursor.execute(
                    "SELECT student_id, student_name, cgpa FROM student_records WHERE dep
                    (dept_name.strip(),)
                )
                return cursor.fetchall()  # Returns list of tuples
            except sqlite3.Error as err:
                raise DatabaseOperationError("Data query operation failed.", err)
            finally:
                if conn:
                    conn.close()

    # --- Execution Driver ---
    if __name__ == "__main__":
        db_app = StudentDatabaseManager("college_demo.db")

        try:
            db_app.initialize_schema()
            db_app.add_student(101, "Aditi Sharma", "CSE", 8.7)
            db_app.add_student(102, "Rahul Verma", "CSE", 7.9)

            cse_students = db_app.fetch_students_by_dept("CSE")
            print("\n--- CSE Department Records ---")
            for record in cse_students:
                print(f"ID: {record} | Name: {record} | CGPA: {record}")

        except ValueError as ex:
            print(f"[Validation Error] {ex}")
        except DatabaseOperationError as ex:
            print(f"[Database Error] {ex}")
        finally:
            print("[Execution] Database session complete.")
```

Question 2 (20 Marks): NumPy Vectorized Computation & Scalar Array Manipulation

(a) [5 Marks] Theoretical Concepts

1. Contrast element-wise array operations in NumPy against Python list iteration.
2. Explain the execution outcome of multiplying a Python list by an integer (`* 2`) versus multiplying a NumPy array by a scalar (`np.array() * 2`).

(b) [5 Marks] Array Inspection & Aggregation

List 4 built-in NumPy array methods/attributes (e.g., `.shape` , `.dtype` , `np.mean()` , `np.sum()`).

© [5 Marks] Simple Baseline Solution: NumPy Vectorized Grading System

```
import numpy as np

# Creating NumPy 1D and 2D Arrays
raw_scores = np.array()

# Vectorized Arithmetic Operations
curved_scores = raw_scores + 5  # Adds 5 to all elements in parallel

# Boolean Mask Filtering
passing_filter = curved_scores >= 50
passed_students = curved_scores[passing_filter]

print("Original Scores:", raw_scores)
print("Curved Scores  :", curved_scores)
print("Passed Scores  :", passed_students)
print("Class Mean     :", np.mean(curved_scores))
```

(d) [5 Marks] Complex Improvement: OOP Vectorized Matrix Processor with Array Bounds Checking

```
import numpy as np

# --- Custom Matrix Exception ---
class MatrixDimensionError(Exception):
    """Custom exception raised when array dimensions are incompatible."""
    def __init__(self, shape_a, shape_b):
        self.shape_a = shape_a
        self.shape_b = shape_b
        super().__init__(f"Dimension Mismatch: Array shape {shape_a} is incompatible
```

```
        # --- Modular OOP Array Processor ---
        class StatisticalMatrixEngine:
            def __init__(self, data_matrix):
                self.matrix = None
                self.load_matrix(data_matrix)

            def load_matrix(self, raw_input):
                """Converts raw list input into a validated NumPy float ndarray."""
                try:
                    arr = np.array(raw_input, dtype=np.float64)
                    if arr.size == 0:
                        raise ValueError("Input dataset cannot be empty.")
                    self.matrix = arr
                except (ValueError, TypeError) as err:
                    raise ValueError(f"Array conversion failed: {err}")

            def normalize_matrix(self):
                """Computes min-max normalized array using vectorized operations."""
                min_val = np.min(self.matrix)
                max_val = np.max(self.matrix)

                range_val = max_val - min_val
                if range_val == 0:
                    raise ZeroDivisionError("Cannot normalize uniform array (max - min = 0).'

                # Vectorized scalar normalization formula
                return (self.matrix - min_val) / range_val

            def add_matrix(self, other_engine):
                """Performs element-wise array addition with matrix shape validation."""
                if not isinstance(other_engine, StatisticalMatrixEngine):
                    raise TypeError("Argument must be an instance of StatisticalMatrixEngine.

                if self.matrix.shape != other_engine.matrix.shape:
                    raise MatrixDimensionError(self.matrix.shape, other_engine.matrix.shape)

                return self.matrix + other_engine.matrix

            def get_summary_stats(self):
                return {
                    "shape": self.matrix.shape,
                    "dimensions": self.matrix.ndim,
                    "mean": round(float(np.mean(self.matrix)), 2),
                    "std_dev": round(float(np.std(self.matrix)), 2)
                }
```

```
# Driver Execution
if __name__ == "__main__":
    matrix_a = [,]
    matrix_b = [,]

    try:
        engine_a = StatisticalMatrixEngine(matrix_a)
        engine_b = StatisticalMatrixEngine(matrix_b)

        print("--- Matrix A Summary ---")
        print(engine_a.get_summary_stats())

        sum_result = engine_a.add_matrix(engine_b)
        print("\n--- Vectorized Sum Result Matrix ---")
        print(sum_result)

        normalized = engine_a.normalize_matrix()
        print("\n--- Normalized Matrix A ---")
        print(np.round(normalized, 3))

    except (ValueError, TypeError, ZeroDivisionError) as ex:
        print(f"[Computation Error] {ex}")
    except MatrixDimensionError as ex:
        print(f"[Matrix Dimension Failure] {ex}")
```

Question 3 (20 Marks): Pandas DataFrames, Data Cleaning, and Group Aggregation

(a) [5 Marks] Theoretical Concepts

1. Define the structural differences between a Pandas **Series** and a **DataFrame**.
2. Explain how Pandas handles missing data using `.isna()`, `.dropna()`, and `.fillna()`.

(b) [5 Marks] Selection & Slicing

Differentiate between `.loc[]` (label-based indexing) and `.iloc[]` (integer position-based indexing) in a Pandas DataFrame.

© [5 Marks] Simple Baseline Solution: DataFrame Creation & Aggregation

```
import pandas as pd
```

```
# Creating DataFrame from Dictionary
data = {
    "StudentID":,
    "Name": ["Alpha", "Beta", "Gamma", "Delta"],
    "Department": ["CSE", "ECE", "CSE", "ECE"],
    "Score": [88.5, 92.0, 75.5, 64.0]
}

df = pd.DataFrame(data)

# Filtering Rows
cse_df = df[df["Department"] == "CSE"]

# Grouped Mean Aggregation
dept_summary = df.groupby("Department")["Score"].mean()

print("--- Filtered CSE DataFrame ---")
print(cse_df)
print("\n--- Mean Score by Department ---")
print(dept_summary)
```

(d) [5 Marks] Complex Improvement: OOP Data Cleaning Pipeline Class with Type Validation

```
import pandas as pd
import numpy as np

# --- Custom Data Processing Exception ---
class DatasetValidationError(Exception):
    """Raised when DataFrame fails column presence or data integrity checks."""
    def __init__(self, missing_cols):
        self.missing_cols = missing_cols
        super().__init__(f"Dataset Validation Failed: Missing required columns: {miss

# --- Modular OOP Data Processor Class ---
class DataPipelineProcessor:
    REQUIRED_COLUMNS = {"Department", "Score", "Attendance"}

    def __init__(self, raw_dataframe):
        self.df = None
        self.validate_and_load(raw_dataframe)

    def validate_and_load(self, raw_df):
        if not isinstance(raw_df, pd.DataFrame):
            raise TypeError("Input data must be a valid Pandas DataFrame.")
```

```
            missing = DataPipelineProcessor.REQUIRED_COLUMNS - set(raw_df.columns)
            if missing:
                raise DatasetValidationError(list(missing))

            self.df = raw_df.copy()

        def clean_missing_values(self):
            """Imputes missing numeric scores with mean and drops remaining nulls."""
            if self.df["Score"].isna().any():
                mean_score = self.df["Score"].mean()
                self.df["Score"] = self.df["Score"].fillna(mean_score)

            # Drop any remaining null rows
            self.df = self.df.dropna()

        def generate_department_analytics(self):
            """Computes summary metrics grouped by Department."""
            self.clean_missing_values()

            summary_df = self.df.groupby("Department").agg(
                StudentCount=("Score", "count"),
                AverageScore=("Score", "mean"),
                MaxAttendance=("Attendance", "max")
            ).reset_index()

            return summary_df

    # Driver Execution
    if __name__ == "__main__":
        sample_data = {
            "StudentID":,
            "Department": ["CSE", "ECE", "CSE", "ECE", "CSE"],
            "Score": [85.0, np.nan, 78.0, 90.0, 92.0],  # Contains NaN
            "Attendance":
        }

        raw_df = pd.DataFrame(sample_data)

        try:
            processor = DataPipelineProcessor(raw_df)
            analytics = processor.generate_department_analytics()

            print("--- Department Analytics Summary ---")
            print(analytics.to_string(index=False))

        except TypeError as ex:
```

```
        print(f"[Type Error] {ex}")
    except DatasetValidationError as ex:
        print(f"[Validation Error] {ex}")
```

Question 4 (20 Marks): SQLite3 & Pandas Data Ingestion Integration

(a) [5 Marks] Theoretical Concepts

1. Describe how `pd.read_sql_query()` executes an SQL query directly over an active `sqlite3` connection handle and outputs a Pandas DataFrame.
2. Explain how `df.to_sql()` writes Pandas DataFrame tables back to an SQLite database file.

(b) [5 Marks] Parameterized Integration Safety

Explain how to pass parameters safely into `pd.read_sql_query()` using its `params` argument to prevent SQL injection vulnerabilities.

© [5 Marks] Simple Baseline Solution: SQLite to DataFrame Reader

```
import sqlite3
import pandas as pd

# Connect to database file
conn = sqlite3.connect("college.db")

# Ingest table directly into Pandas DataFrame
query = "SELECT * FROM students WHERE branch = ?"
df = pd.read_sql_query(query, conn, params=("CSE",))

# Close connection handle
conn.close()

print("Ingested DataFrame:")
print(df.head())
```

(d) [5 Marks] Complex Improvement: OOP SQLite-Pandas Ingestion Engine with Exception Guards

```
import sqlite3
import pandas as pd
```

```
# --- Custom Exception ---
class DataIntegrationError(Exception):
    def __init__(self, message, cause=None):
        super().__init__(f"Data Integration Failed: {message} | Cause: {cause}")

# --- Integrated Database-DataFrame Engine ---
class DatabaseDataFrameBridge:
    def __init__(self, db_path):
        self.db_path = db_path

    def get_connection(self):
        return sqlite3.connect(self.db_path)

    def export_dataframe_to_sqlite(self, df, table_name):
        """Exports a Pandas DataFrame into an SQLite table."""
        conn = None
        try:
            if not isinstance(df, pd.DataFrame):
                raise TypeError("First argument must be a Pandas DataFrame.")

            conn = self.get_connection()
            # Export DataFrame to SQL table
            df.to_sql(table_name, conn, if_exists="replace", index=False)
            conn.commit()
            print(f"[Bridge] DataFrame exported to table '{table_name}' successfully

        except sqlite3.Error as err:
            raise DataIntegrationError("Failed to write DataFrame to SQLite.", err)
        finally:
            if conn:
                conn.close()

    def import_filtered_query(self, table_name, min_cgpa):
        """Reads database rows safely into DataFrame using parameterized query."""
        conn = None
        try:
            conn = self.get_connection()
            # Safe Parameterized SQL Read
            sql_query = f"SELECT * FROM {table_name} WHERE cgpa >= ?"
            filtered_df = pd.read_sql_query(sql_query, conn, params=(float(min_cgpa),
            return filtered_df

        except sqlite3.Error as err:
            raise DataIntegrationError("Failed to read SQL query into DataFrame.", er
        finally:
            if conn:
```

```
                conn.close()

# Execution Driver
if __name__ == "__main__":
    # Create sample DataFrame
    initial_data = pd.DataFrame({
        "roll_no": ,
        "name": ["Aman", "Bhavna", "Chetan", "Deepa"],
        "branch": ["CSE", "ECE", "CSE", "MECH"],
        "cgpa": [8.5, 7.2, 9.1, 6.8]
    })

    bridge = DatabaseDataFrameBridge("academic_bridge.db")

    try:
        bridge.export_dataframe_to_sqlite(initial_data, "college_students")
        retrieved_df = bridge.import_filtered_query("college_students", min_cgpa=8.0)

        print("\n--- High-Achieving Students (CGPA >= 8.0) ---")
        print(retrieved_df.to_string(index=False))

    except DataIntegrationError as ex:
        print(f"[Integration Error] {ex}")
    except TypeError as ex:
        print(f"[Type Error] {ex}")
```

Question 5 (20 Marks): Capstone — Desktop GUI Analytics Application (Tkinter + SQLite3 + Pandas + Exception Handling)

(a) [5 Marks] Desktop GUI & Database Architecture

Explain the role of `Tkinter` as a frontend user interface, `sqlite3` for local persistent disk storage, and `Pandas` as an analytics processing engine.

(b) [10 Marks & © [5 Marks] Complete Capstone Application: Student Database & Analytics GUI

This continuous OOP application integrates `Tkinter` input widgets, custom exception classes (`InvalidCGPAError`), parameterized `sqlite3` persistence, and `Pandas` DataFrame aggregation.

```
import tkinter as tk
from tkinter import messagebox
import sqlite3
```

```
import pandas as pd

# --- Custom Exception Class ---
class InvalidCGPAError(Exception):
    def __init__(self, cgpa_val):
        super().__init__(f"Invalid CGPA '{cgpa_val}'. Must be a floating point numbe

# --- SQLite Database Storage Layer ---
class StudentDatabaseLayer:
    def __init__(self, db_name="capstone_students.db"):
        self.db_name = db_name
        self.init_db()

    def get_conn(self):
        return sqlite3.connect(self.db_name)

    def init_db(self):
        conn = None
        try:
            conn = self.get_conn()
            cursor = conn.cursor()
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS students (
                    roll INTEGER PRIMARY KEY,
                    name TEXT NOT NULL,
                    branch TEXT NOT NULL,
                    cgpa REAL NOT NULL
                )
            ''')
            conn.commit()
        finally:
            if conn:
                conn.close()

    def insert_student(self, roll, name, branch, cgpa):
        conn = None
        try:
            conn = self.get_conn()
            cursor = conn.cursor()
            cursor.execute("INSERT INTO students VALUES (?, ?, ?, ?)", (roll, name, b
            conn.commit()
        except sqlite3.IntegrityError:
            raise ValueError(f"Roll No {roll} already exists in database.")
        finally:
            if conn:
                conn.close()
```

```
    def get_all_as_dataframe(self):
        conn = None
        try:
            conn = self.get_conn()
            return pd.read_sql_query("SELECT * FROM students", conn)
        finally:
            if conn:
                conn.close()

# --- Main GUI Application ---
class StudentAnalyticsApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Student Persistence & Analytics Studio")
        self.geometry("540x440")
        self.config(bg="#f4f6f7")

        self.db = StudentDatabaseLayer()
        self.build_ui()
        self.refresh_listbox()

    def build_ui(self):
        # Header Banner
        header = tk.Frame(self, bg="#2c3e50", pady=10)
        header.pack(side=tk.TOP, fill=tk.X)
        tk.Label(header, text="Student Database Analytics Studio", font=("Arial", 14,

        # Input Form Panel
        form_frame = tk.LabelFrame(self, text=" Student Record Entry ", font=("Arial'
        form_frame.pack(pady=10, fill=tk.X, padx=15)

        tk.Label(form_frame, text="Roll No:", bg="#f4f6f7").grid(row=0, column=0, st:
        self.ent_roll = tk.Entry(form_frame, width=12)
        self.ent_roll.grid(row=0, column=1, pady=3, padx=5)

        tk.Label(form_frame, text="Name:", bg="#f4f6f7").grid(row=0, column=2, stick
        self.ent_name = tk.Entry(form_frame, width=12)
        self.ent_name.grid(row=0, column=3, pady=3, padx=5)

        tk.Label(form_frame, text="Branch:", bg="#f4f6f7").grid(row=1, column=0, sti
        self.ent_branch = tk.Entry(form_frame, width=12)
        self.ent_branch.grid(row=1, column=1, pady=3, padx=5)

        tk.Label(form_frame, text="CGPA:", bg="#f4f6f7").grid(row=1, column=2, stick
        self.ent_cgpa = tk.Entry(form_frame, width=12)
```

```
        self.ent_cgpa.grid(row=1, column=3, pady=3, padx=5)

        btn_add = tk.Button(form_frame, text="Save Record", command=self.handle_save,
        btn_add.grid(row=0, column=4, rowspan=2, padx=10, sticky="nesw")

        # Listbox Display Area
        list_frame = tk.Frame(self, bg="#f4f6f7")
        list_frame.pack(pady=5, fill=tk.BOTH, expand=True, padx=15)

        self.scrollbar = tk.Scrollbar(list_frame, orient=tk.VERTICAL)
        self.listbox = tk.Listbox(list_frame, font=("Courier", 9), yscrollcommand=se
        self.scrollbar.config(command=self.listbox.yview)

        self.listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Footer Actions
        footer = tk.Frame(self, bg="#f4f6f7")
        footer.pack(side=tk.BOTTOM, fill=tk.X, pady=10, padx=15)

        tk.Button(footer, text="Run Pandas Summary Analytics", command=self.handle_ar

    def handle_save(self):
        """Validates entry fields, triggers exception checks, and writes to database
        try:
            r_str = self.ent_roll.get().strip()
            name = self.ent_name.get().strip()
            branch = self.ent_branch.get().strip()
            c_str = self.ent_cgpa.get().strip()

            if not r_str or not name or not branch or not c_str:
                raise ValueError("All input fields are required.")

            roll = int(r_str)
            cgpa = float(c_str)

            if cgpa < 0.0 or cgpa > 10.0:
                raise InvalidCGPAError(cgpa)

            # Insert into database layer
            self.db.insert_student(roll, name, branch, cgpa)

            # Reset Form and Refresh Listbox
            self.ent_roll.delete(0, tk.END)
            self.ent_name.delete(0, tk.END)
            self.ent_branch.delete(0, tk.END)
```

```
            self.ent_cgpa.delete(0, tk.END)
            self.refresh_listbox()

            messagebox.showinfo("Success", f"Student {name} saved successfully!")

        except ValueError as ex:
            messagebox.showerror("Input Format Error", str(ex))
        except InvalidCGPAError as ex:
            messagebox.showwarning("CGPA Range Error", str(ex))

    def refresh_listbox(self):
        """Loads database data into DataFrame and refreshes UI Listbox."""
        self.listbox.delete(0, tk.END)
        df = self.db.get_all_as_dataframe()
        for _, row in df.iterrows():
            line = f"Roll: {int(row['roll']):<5} | Name: {row['name']:<12} | Branch:
            self.listbox.insert(tk.END, line)

    def handle_analytics(self):
        """Generates statistical summary by Branch using Pandas aggregation."""
        df = self.db.get_all_as_dataframe()
        if df.empty:
            messagebox.showinfo("Analytics Summary", "Database is currently empty.")
            return

        summary = df.groupby("branch").agg(
            StudentCount=("roll", "count"),
            AverageCGPA=("cgpa", "mean")
        ).reset_index()

        msg = "--- Pandas Department Analytics ---\n\n" + summary.to_string(index=Fa
        messagebox.showinfo("Branch Analytics Report", msg)

if __name__ == "__main__":
    app = StudentAnalyticsApp()
    app.mainloop()
```

Key Exam Takeaways Summary Table

Component	Essential Syntax & Operational Rules

Primary Exam Focus

`sqlite3`

`connect()`, `cursor()`, `execute()`, `commit()`, `close()`. Always use `?` placeholders for security.

Schema creation, parameterized insert/select, `try/except sqlite3.Error/finally` connection cleanup.

`NumPy`

`np.array()`, element-wise vector operations, `np.mean()`, `np.std()`.

Vectorized math without loops, broadcasting, array dimension validation.

`Pandas`

`pd.DataFrame()`, `.groupby()`, `.fillna()`, `.dropna()`, `.loc[]`, `.iloc[]`.

Data cleaning, group aggregation, tabular data transformations.

Integration

`pd.read_sql_query(sql, conn, params=())`, `df.to_sql(table, conn)`.

Passing database tables safely to/from Pandas DataFrames.